MARJON SEALZA

LIFE & STORE — Complete Master Plan

Version 3 • Consolidated scope, professional-site standards, PayMongo, security, operations, and release gates

Purpose: One implementation specification for the existing Marjon Sealza website and online store. This document consolidates the previous plan and the additional recommendations into one prioritized master checklist.

Important status note: This is a specification and acceptance checklist, not proof that each feature has been implemented. Changes must be made to the existing project, validated, and deployed without replacing it with an unrelated or duplicate site.

Primary target: marjonsealza.life and www.marjonsealza.life. Existing Cloudflare Pages project/repository should remain the source of truth unless an explicit migration is approved.

1. Product direction and non-negotiable rules

- Keep the existing website and brand; extend the current codebase instead of creating a duplicate site or deploying a separate demo over the production domain.
- Keep portfolio/personal brand pages and LIFE & STORE connected through clear navigation, consistent design, mobile responsiveness, and shared branding.
- Use npm as the project package manager. Keep package.json and package-lock.json synchronized; remove conflicting Bun lockfiles. Required release command: npm run build.
- Never expose PayMongo secret keys, webhook secrets, database credentials, or administrative credentials in frontend code, browser storage, public repositories, logs, or customer-facing pages.
- Do not claim checkout, payment, email, deployment, or order fulfillment works until the relevant flow has been tested with recorded results.
- Use PayMongo's official checkout/payment flow and supported payment methods. Do not imitate GCash QR codes or present a generated QR as a valid GCash payment request.
- Maintain a clear test/live separation. Test mode must never be mistaken for real-money production processing.

2. Public website and portfolio

Core pages

- Home/landing page with a clear value proposition, professional introduction, primary calls to action, and links to portfolio and store.
- About/profile page with concise biography, skills, experience, and relevant credibility signals.
- Portfolio/projects page with project cards, descriptions, screenshots where available, technology used, and working links.
- Services page with service descriptions, deliverables, process, expected timelines, and inquiry CTA.
- Contact page with validated contact form, success/error states, spam protection, and accessible contact alternatives.
- LIFE & STORE product catalog, product detail pages, cart, checkout, order confirmation, and customer support.
- Custom not-found page and helpful empty, loading, offline, and error states.

Professional quality standards

- Responsive layouts for small Android phones, tablets, laptops, and wide desktop screens; avoid horizontal overflow.
- Consistent typography, spacing, colors, button states, form styling, and navigation behavior.
- Accessibility: semantic headings, keyboard navigation, visible focus, labels, sufficient contrast, descriptive image alt text, and reduced-motion support.
- Performance: optimized/compressed images, lazy loading below the fold, minimal JavaScript, caching where safe, and no unnecessary heavy animation.
- SEO: unique page titles and descriptions, canonical URLs, Open Graph/social previews, sitemap.xml, robots.txt, meaningful headings, structured data where accurate, and human-readable URLs.
- Favicon/app icons, branded 404 page, clean footer, and clear privacy/contact links.
- No fake testimonials, fabricated reviews, unsupported guarantees, broken placeholder buttons, or misleading payment/security claims.

3. LIFE & STORE commerce requirements

Catalog and cart

- Product listing with product name, image, price in PHP, short description, availability, and category/filter/search where useful.

- Product details with full description, variations/options when applicable, shipping/fulfillment notes, and clear stock/availability status.
- Cart supports quantity changes, remove item, subtotal, shipping/fees if applicable, and final total before payment.
- Validate prices and quantities on the server. Never trust totals, discounts, or product prices sent only by the browser.
- Prevent checkout for unavailable items and revalidate stock/price at order creation.
- Use clear empty-cart, validation, loading, and retry states; retain cart safely where practical without storing sensitive payment data.

Checkout and order lifecycle

- Collect only necessary customer details: name, email/phone, delivery or fulfillment details when needed, and order notes if appropriate.
- Display itemized order summary, total in PHP, payment method options, privacy notice, and customer support route before the customer pays.
- Create a unique internal order ID and reference number before redirecting to payment.
- Record pending, paid, failed, cancelled, and refunded states; store timestamps and provider identifiers without storing card data.
- Build customer-facing success, cancellation, failure, and pending pages. A redirect alone must never mark an order as paid.
- Make order creation and webhook processing idempotent to prevent duplicate orders or duplicate fulfillment.
- Send order confirmation only after the backend confirms the appropriate state; clearly distinguish payment initiated from payment confirmed.

4. PayMongo integration

Secure architecture

- Browser calls a server-side checkout endpoint; server validates the cart against trusted product/price data and creates the PayMongo checkout/payment session.
- PayMongo secret API key is stored only as a server-side secret. Never place it in Vite variables prefixed for public exposure or in client-side bundles.
- Use the configured backend/Worker and D1 store_orders table as the authoritative order state. Keep the existing database and production app aligned.
- Set the correct success and cancel return URLs for the official domain and validate all incoming parameters.
- Use only payment methods enabled for the merchant's PayMongo account and supported by the current integration; availability can depend on account approval and configuration.
- Use a server-side webhook endpoint to verify PayMongo's signature using the exact raw request body and configured webhook secret, with timestamp/replay protections where supported.
- Acknowledge valid webhook events promptly; process events idempotently, log event IDs, and handle retries safely.
- Update an order to paid only after a verified provider event or a trusted server-side payment-status confirmation. Validate amount, currency, reference, session/order association, and live/test mode.
- Handle delayed payment, failed payment, cancellation, duplicate events, webhook retries, refund events, and out-of-order events safely.
- Never trust a client redirect or query-string status as proof of payment. Never store full card numbers, CVV, or sensitive payment credentials.

Payment method display

- Show only the methods that are genuinely enabled and available for the merchant, such as GCash, QRPh, cards, Maya, or other supported wallets when approved.
- Use the PayMongo-hosted checkout experience or the supported API flow. Do not generate unofficial QR codes or claim that an ERP-generated QR is recognized by GCash.
- Provide clear instructions for returning from the provider and what to do if the payment status remains pending.

Configuration and go-live checks

- Confirm the Worker has the correct PAYMONGO_MODE, APP_BASE_URL, D1 binding, API key secret, and webhook signing secret configured.
- Register the exact webhook URL in the PayMongo dashboard and confirm the configured webhook secret matches the Worker secret.
- Verify health/readiness endpoint reports ready only when required configuration exists; do not expose secret values in health responses.
- Run end-to-end tests in test mode first; only switch to live mode after account approval, webhook verification, refund/support process, and test evidence are complete.
- Document how to rotate keys, disable checkout during an incident, inspect failed events, and reconcile provider transactions with local orders.

5. Backend, data, and order administration

- Define server-side API contracts for products, checkout creation, order status, webhook intake, and admin order management.
- Validate inputs, enforce request size limits, return safe error messages, and avoid leaking stack traces or secrets.
- Use D1 migrations or versioned schema changes; keep schema changes reproducible and documented.
- Add useful order fields as needed: customer contact, line-item snapshot, subtotal, shipping, discounts, total, currency, payment provider, payment status, created/updated timestamps, and fulfillment status.
- Store an immutable snapshot of product names/prices at purchase time so historical orders remain accurate after catalog edits.
- Separate payment status from fulfillment status; a paid order can still be unfulfilled, shipped, completed, or cancelled according to policy.
- Add admin search/filter by order number, date, status, and customer; protect all admin endpoints with authentication and role-based authorization.
- Provide a basic audit trail for status changes and admin actions; never permit unauthenticated public access to customer/order records.
- Create backup/export and restore procedures appropriate to the chosen database and retention needs.

6. Admin and store operations

- Secure admin sign-in, session expiry, sign-out, rate limiting, and least-privilege access. Do not rely on a hidden URL as security.
- Product management: create/edit/archive products, upload/replace images safely, manage prices, descriptions, stock, categories, and visibility.
- Order management: view order details, inspect verified payment state, update fulfillment, add internal notes, and issue refunds only through an authorized supported process.
- Dashboard metrics: orders by status, gross sales, refunds, average order value, best-selling products, and payment reconciliation differences.

- CSV export for order records with explicit date range and permission checks; escape spreadsheet-formula injection in exported fields.
- Show system health and recent errors without displaying API secrets, customer passwords, or unnecessary personal data.

7. Customer trust, legal, privacy, and support

- Publish accurate Privacy Policy, Terms of Service, Shipping/Delivery or Fulfillment Policy, Returns/Refunds/Cancellation Policy, and Contact/Support information.
- Explain what personal information is collected, why it is collected, retention, sharing with payment providers, and how customers can request help or exercise applicable privacy rights.
- Collect only data needed to process and support orders; restrict access, define retention periods, and securely delete data when appropriate.
- Use consent and disclosure text for optional marketing messages; transactional notifications must be clearly separated from promotional marketing.
- Provide a support email/contact form and a clear process for payment pending, duplicate payment, failed payment, cancellation, refund, and undelivered order cases.
- Use truthful delivery timelines, stock descriptions, pricing, refund rules, and customer-facing payment status wording.
- Review Philippine legal, tax, consumer-protection, and data-privacy obligations applicable to the actual business before launch; this document is not legal or tax advice.

8. Notifications and customer communication

- Order received notification after the order is created, stating that payment may still be pending.
- Payment confirmation only after verified payment; include order/reference number, amount, and support contact.
- Failure/cancellation/pending instructions with a safe retry path that does not create accidental duplicate orders.
- Fulfillment/shipping notification only when the relevant status is actually updated.
- Use an email provider or other supported channel configured server-side; do not promise email/SMS delivery until credentials, domain verification, and delivery tests pass.
- Avoid sending sensitive payment information through email/SMS. Log delivery outcome and provide an admin-visible retry mechanism where appropriate.

9. Security, abuse prevention, and reliability

- HTTPS everywhere; keep TLS and custom-domain configuration valid for both apex and www hostnames.
- Server-side validation, output escaping, safe error handling, and protection against common injection and cross-site scripting risks.
- Rate-limit checkout creation, contact forms, sign-in, and webhook endpoints appropriately; add bot/spam mitigation to public forms.
- Use secure, HTTP-only, SameSite cookies for sessions if cookie-based auth is used; protect state-changing admin requests against CSRF where applicable.
- Apply least-privilege access to Cloudflare, GitHub, database, payment dashboard, and deployment credentials; remove obsolete tokens and rotate exposed credentials.
- Keep dependencies updated and run audit/build checks; avoid unmaintained or unnecessary packages.
- Use structured logs with correlation IDs while redacting secrets, payment credentials, and unnecessary personal data.
- Plan for downtime: clear maintenance/error messages, safe checkout disable switch, retries, provider outage handling, and incident notes.

- Do not use test payment keys or test-mode sessions in live checkout, and do not mix test and live orders.

10. Analytics, discovery, and conversion

- Track privacy-respecting, useful events such as product views, add-to-cart, checkout start, successful purchase, and checkout errors.
- Exclude card/payment secrets and unnecessary personal information from analytics events.
- Add search engine indexing controls, sitemap, social previews, and accurate structured data only where the content supports it.
- Measure mobile performance, conversion drop-off, top products, failed checkout causes, and support requests; use the findings to prioritize improvements.
- Add a clear call to action and visible support path without intrusive popups, deceptive urgency, or fake scarcity.

11. Testing and acceptance criteria

Build and code quality

- Install from a clean checkout using npm ci; ensure package-lock.json is synchronized with package.json.
- Run npm run build and fix all errors before release; run lint/type checks and automated tests if configured.
- Check for console errors, broken links, missing assets, layout overflow, keyboard accessibility, and form validation failures.
- Test production build locally or in preview before publishing.

End-to-end payment scenarios

- Valid cart creates one pending order and one provider checkout session.
- Tampered price/quantity, invalid product, zero/negative quantity, and malformed requests are rejected server-side.
- Successful verified webhook marks the correct order paid exactly once.
- Invalid signature, missing secret, duplicate event, wrong amount/currency, unknown order, and test/live mismatch do not mark an order paid.
- Cancelled, failed, delayed, retried, refunded, and out-of-order events are handled without corrupting order state.
- Success-page refresh or repeated browser redirects do not duplicate orders or mark unpaid orders paid.
- Checkout is tested on Android mobile and desktop across supported browsers and realistic network conditions.
- Admin access is denied to unauthenticated users; users cannot access orders outside their authorized scope.

Release gate

- No critical/high-severity security issue remains unresolved.
- Build passes and the deployment status is confirmed successful.
- Smoke tests pass on the exact deployed URL and both custom domains.
- PayMongo webhook signature and a complete test payment are verified with evidence.
- Monitoring, support contacts, rollback/disable procedure, and operator instructions are documented.
- Live mode is enabled only after explicit readiness approval and the merchant's PayMongo account/configuration is ready.

12. Deployment and release operations

- Keep the existing Cloudflare Pages project and GitHub repository as the production source of truth; avoid deploying a separate starter or demo to the custom domain.

- Use npm, npm run build, and dist/ as the Pages output directory where that remains the current project configuration.
- Use environment variables for public non-secret configuration and encrypted/server-side secrets for private keys.
- Deploy through a branch/preview workflow where possible; review changes and run checks before merging to production.
- After deployment, verify HTTP status, asset loading, console/network errors, domain redirects, HTTPS certificates, checkout endpoint, Worker health, database access, and webhook delivery.
- Keep a rollback path to the last known-good deployment and document how to disable checkout quickly without taking down the portfolio.
- Do not call the project complete solely because the homepage loads. Checkout, webhook, order state, admin, and customer support flows must be verified.

13. Suggested implementation sequence

Priority	Workstream	Completion evidence
P0	Inspect and protect current production app	Source/config confirmed; no duplicate app replaces domain
P0	Server checkout endpoint and trusted pricing	Valid checkout creates one pending order/session
P0	PayMongo keys + webhook secret + D1 binding	Readiness checks pass without exposing secrets
P0	Webhook verification and order-state updates	Signature tests and idempotency tests pass
P0	Customer success/cancel/failure/pending states	Redirect cannot fake payment confirmation
P0	Clean npm build and deployment smoke tests	npm ci + npm run build + deployed URL checks pass
P1	Catalog, cart, product detail, order history/support	Mobile and desktop checkout journey passes
P1	Secure admin order/product workflow	Authentication, authorization, audit trail tested
P1	Policies, support, transactional notifications	Accurate pages and delivery tests confirmed
P2	SEO, analytics, performance, accessibility polish	Audits and measurable baseline recorded
P2	Reports, reconciliation, backup/restore	Export and recovery procedures tested

14. Final sign-off checklist

- The existing portfolio and LIFE & STORE both work and remain visually consistent.
- All visible links/buttons have real destinations/actions; no placeholder interactions remain.
- Product pricing, cart totals, stock checks, and order references are enforced server-side.
- PayMongo secrets are stored securely and webhook signature verification is confirmed.
- Payment state is based on verified backend/provider evidence, not a redirect.
- Order, payment, refund, and fulfillment states are clear and auditable.
- Policies, support details, notifications, privacy, and analytics are accurate and tested.
- Clean npm install and npm run build pass; production deployment and custom domains are smoke-tested.

- No test-mode payment is confused with a live payment; live checkout is enabled only when explicitly ready.
- A rollback and checkout-disable procedure is known and tested.

Definition of done: The website is complete only when the implemented features are deployed to the intended production project, the build succeeds, and the critical customer journey—including a verified PayMongo payment event and the resulting order status—is tested end to end. Any blocked item must be clearly marked as blocked, not represented as complete.

Prepared as a consolidated implementation specification. Confirm actual platform, merchant-account, and legal requirements before production use.

MASTER PLAN v4 — ADDENDUM

PayMongo integration moved to the To-Do list

Decision recorded: skip PayMongo implementation for now. Do not block the rest of the website/store work on payment-gateway setup. PayMongo remains a deferred task and must not be described as implemented, production-ready, or verified until the checklist below passes.

Deferred task checklist

- 1. Configure Cloudflare Worker secrets for the existing Worker **marjonsealza-paymongo**: **PAYMONGO_SECRET_KEY**, **PAYMONGO_WEBHOOK_SECRET**, and **ADMIN_API_TOKEN**. Use test-mode credentials first. Keep secret values in Cloudflare; never put them in source control or chat.
- 2. Verify the Worker health endpoint reports readiness. A successful HTTP 200 alone is not enough; confirm the response indicates **ready: true**.
- 3. Confirm PayMongo test-mode checkout session creation, supported payment methods, success/cancel/failure return URLs, and safe error handling.
- 4. Configure and verify the PayMongo webhook signing secret; validate signatures, reject invalid events, and make event processing idempotent.
- 5. Verify order/payment status updates in the existing Cloudflare D1 database and audit logs. Test duplicate, delayed, failed, and out-of-order webhook events.
- 6. Run end-to-end test transactions in test mode only. Verify the customer journey and admin/order workflow without charging real money.
- 7. Recheck security: admin token authorization, no secret leakage in logs or frontend bundles, HTTPS-only routes, and appropriate rate/error handling.
- 8. Run **npm run build**, automated tests, production health checks, and deployment verification. Do not mark the store launch-ready until the build and complete payment flow are verified.

Current status at the time of this addendum

The public Worker health endpoint returned HTTP 200 with `ready: false` and `mode: test`. The Worker secret listing previously returned no configured secrets. The production website was not changed during these checks. The source archive's automated tests passed, but the production build and complete payment flow remain unverified.

Implementation rule

Continue other Master Plan tasks independently. Keep the existing official website and domain in place; do not replace production with a separate prototype. Treat PayMongo as a pending integration until all deferred checklist items have been completed and verified.